# Data-Structures & Algorithms

*with*

Manoj Prabhakaran

## Lecture 26

## Hash Tables

### Open Addressing

# Hash Tables

- Recall two paradigms for handling collisions:
  - At each index, allow multiple items to be held
    - The bucket holding the colliding items can grow arbitrarily, unless the hash functions are (cryptographically) strong
    - Items are scattered around in memory (linked together in a global linked list), negating cache benefits
  - When collision, find another spot in the hash table
    - Benefits from caching of the hash table
    - While inserting, may need to try multiple times before finding a spot. And while searching, will need to look at all those potential spots.

Closed Addressing

Open Addressing

# Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```

- Inserts at the first free location in the sequence (hence greedy)
- Later, when searching, guaranteed to look till the inserted location (will return early only if a slot that is never used is encountered, which insertion couldn't have skipped)
- If hash sequence is periodic mod m, N=m. If the hash sequence guaranteed to visit all m locations, N = m-#never used slots suffices: after N+1 probes guaranteed to see a never used slot.

# Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```




- To handle deletions, we mark a slot as never used, previously used (deleted), or currently used. Free slots are those which are not currently used.
  - Suppose $key_1$ is inserted at index $t_1$, followed by $key_2$ at index $t_2$. Also, suppose while inserting $key_2$, we encounter $t_1$ before reaching $t_2$.
  - Now suppose $key_1$ is deleted, and then we search for $key_2$. While looking for $key_2$ we should not stop at the free slot $t_1$.

# Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```

- To insert new elements, need the load factor to be < 1 (cf. closed addressing)
- Probing doesn't affect entries already in the hash table (later: probing with reordering)
- Efficiency determined by Probe-Sequence Length: number of iterations of the loop
- Above, inserting *assumed* key is not already present. If key present, should update instead of inserting a new item. Should do this without scanning the array twice.

# Probing

- A more precise template. Initially, every slot in A has `(never_used,-)`.

```
Probe(key): // returns (FOUND,index) or (NOTFOUND,first free slot)
initialise an (implicit) sequence H₀(key), H₁(key), ...
first_free = -1
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if A[t] matches key:  return (FOUND,t)
   if A[t] deleted or never_used: if first_free == -1: first_free = t
   if A[t] never_used: break // end the probe
return (NOTFOUND, first_free)
```

```
Search(key):
(status, slot) = Probe(key)
if status == FOUND: return A[slot]
else return not_found
```

```
Delete(key):
(status, slot) = Probe(key)
if status == FOUND: A[slot] = (deleted,-)
```

Tombstones can slow down probe

```
Insert(item):
(status, slot) = Probe(item.key)
if slot != -1: if A[slot] never_used, ++N; A[slot] = (used,item); return SUCCESS
else return FAILED
```

# Uniform Probing

- Uniform probing: An "ideal" that we shall approximate
  - For each (key,i), $H_i$(key) is a fresh random index
- Expected probe-sequence length:
  - Suppose load factor is $\alpha$ = n/m. Each probe has a probability $\alpha$ of failing (i.e., landing in an occupied slot) and probability (1-$\alpha$) of succeeding.
  - Expected probe-sequence length $= \sum_{i=1}^{\infty} i \times \Pr[\text{took } i \text{ tries}]$

$$= \sum_{i=1}^{\infty} i \times \alpha^{i-1}(1-\alpha) = \frac{1}{1-\alpha}$$

$\Theta\left(\frac{1}{1-\alpha}\right)$ is the best possible for any greedy, non-reordering probing strategy

  - Note: When $\alpha$ approaches 1, expected number of probes spike

# Attendance Break!

Login using CSE LDAP id at

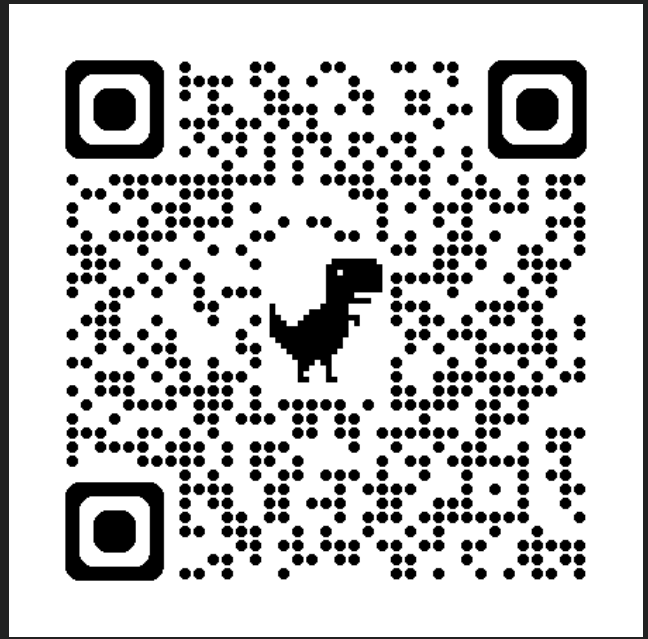
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

**If you are not in the classroom, a report will be sent to DADAC!**



# Linear Probing

- Linear probing: $H_i(\text{key}) = H_0(\text{key}) + i \pmod m$ where $H_0$ is supposed to be a uniform hash
  - Simple: Can compute $H_i(\text{key})$ by simply incrementing (mod m)
  - Cache-friendly: Inspected locations will mostly be in the cache
  - But if $H_i(\text{key}_2)$ runs into $H_0(\text{key}_1)$, then it will have to traverse the segment already traversed by $\text{key}_1$
  - Expected probe-sequence length = $\Theta\left(\left(\frac{1}{1-\alpha}\right)^2\right)$
    - For $\alpha$ close to 1, much larger than in uniform probing